Task 2 — Work Split

for 5 Members

Member 1 - Kenza

The Backend Spokesperson (FastAPI + Rules + Events)

This is the person you correctly identified. They own:

- The event data model — what a decoded event looks like, what a verdict looks like, what a rule looks like.
- The rules engine — the function that takes an event and a rule set and returns a verdict.
- The FastAPI backend — the HTTP and WebSocket endpoints the React app talks to.
- The rule listener thread — receives rule updates from FastAPI and atomically swaps them into the shared rule store.

Why one person for all three: the rules engine and the API are the same contract seen from two sides. Rules come in through the API and end up applied by the engine. If two people wrote these, they'd constantly fight over field names and shapes. One person keeps it coherent.

Deliverable: a rules/ module, an api/ module, and a schemas.py file that defines every data structure used by the program. Other members import from schemas.py — this is the shared vocabulary.

Difficulty: medium-high. This is the most "design" of the roles. Worth noting in the report.

Member 2 - Mehdi

The Spine (Hook + Callback + Message Loop)

They own:

- Loading user32.dll and kernel32.dll through ctypes.
- Declaring all native function signatures and structures.
- Writing the callback function (the one Windows calls).
- Writing the message loop that keeps the program alive.
- Registering and unregistering the hook cleanly.
- Handling the callback's return value so blocking and modifying work.

Why one person: the callback is the single most delicate piece of the program. It runs inside the OS input pipeline, and if it misbehaves the keyboard freezes. It has to be written carefully, tested carefully, and owned by one person who fully understands the timing constraints.

Deliverable: a hook/ module that exposes install_hook(callback) and run_message_loop().

Interface with Member 1: Member 2 writes the callback skeleton and calls functions from Member 1's event module inside it. But Member 2 does not implement decoding or rules — they only call those functions. This keeps the roles clean.

Difficulty: medium. Short code, but every line matters.

Member 3 - Melissa

The Translator (Decode + Context)

They own:

- The keycode-to-name mapping (virtual key codes → human names).
- The modifier state machine — tracking Shift, Ctrl, Alt across events.
- The character reconstruction — turning a key + modifiers into the actual character.
- The context lookup — which window and which process currently has focus.
- Handling edge cases: extended keys, autorepeat, dead keys, unknown keys.

Why one person: decoding is a self-contained problem. It takes a raw event and produces a decoded event. It has nothing to do with rules, logging, or the UI. It can be fully unit-tested offline with a list of fake events — no hook, no VM, no threads needed. That makes it ideal as a standalone role.

Deliverable: a decode/ module that exposes decode(raw_event) -> DecodedEvent and get_context() -> Context.

Interface with Member 1: Member 3 uses the DecodedEvent dataclass defined by Member 1. That's the only contract they need to respect.

Difficulty: medium. Tedious but self-contained.

Member 4 - Lina

The Pipe (Log Queue + Flusher + Persistence)

They own:

- The thread-safe queue that the callback pushes events into.
- The flusher thread that reads the queue and sends events to the FastAPI backend.

- Fallback storage — if the API is down, write to a local file so no events are lost.
- Shutdown flushing — cleanly draining the queue when the program exits.
- The wire format — the exact JSON shape the flusher sends to the API.

Why one person: everything about "moving events out of the callback and into the world" belongs to one person. The queue, the thread, the retry logic, the fallback file, the shutdown sequence — these are all one concern. Splitting them across people would mean constant re-negotiation of buffer sizes and error handling.

Deliverable: a transport/ module with start_flusher(queue, api_url) and push_event(queue, event).

Interface with Member 1: Member 4 and Member 1 jointly define the wire format — the JSON shape the flusher sends to the API. They agree on this once, and both sides code against it.

Difficulty: medium. The core logic is small; the edge cases (API down, queue full, graceful shutdown) take most of the time.

Member 5 - Abdallhadi

The Face (React Frontend)

They own:

- The React app — all UI components.
- The WebSocket client that receives live events from the FastAPI backend.
- The rule editor — form for creating, editing, enabling/disabling rules.
- The live event feed — a scrolling list of recent keyboard events, showing key, character, context, and verdict.
- The mode switch — a toggle for observe-only vs. blocking.
- The connection status — a visible indicator of whether the backend is reachable.

Why one person: the frontend is self-contained. It talks to the API over HTTP + WebSocket. As long as Member 1 publishes the API schema, Member 5 can build the entire UI without waiting on any other member.

Deliverable: a frontend/ React project with a working dev server.

Interface with Member 1: Member 5 consumes the API. They need the route shapes (URLs, request bodies, response bodies, WebSocket message format) — which are Member 1's responsibility to publish early.

Difficulty: medium. React work is familiar but has the most lines of code. Worth noting in the report.